Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM302 Үйлдлийн систем

Linux цөмийн модулиар процессын жагсаалт ба холбоосыг дүрслэх

Linux Kernel Module for Listing Tasks

Бие даалт №1

Судалгаа, хэрэгжилт, туршилтын тайлан

Шалгасан багш: Г.Ганбат

Багийн дугаар: IV-2-3

Гүйцэтгэсэн оюутнууд: А.Тэмүүлэн /B242270063/

Ч.Балгандамба /B232270142/

Д.Азмаа /B241940101/

Н.Билэгт-Очир /B231910009/

Т.Түвшинтөр /B190910820/

Б.Өсөхбаяр /B210930901/

Улаанбаатар хот

2026 он

Гарчиг

Хураангуй **1**

1 Удиртгал **1**

2 Сурах бичгийн шаардлага **1**

3 Онолын үндэс **2**

4 Системийн зохиомж **3**
4.1 Snapshot унших дараалал ба интерфэйс 4

5 Хэрэгжилт **5**

6 Туршилт ба UI дүрслэл **5**

7 Туршилтын арга **6**

8 Үр дүн **7**
8.1 ps командтай харьцуулсан үр дүн 8
8.2 Процесс ба thread-ийн граф 9
8.3 Төлөвийн өөрчлөлт 10

9 Хэлэлцүүлэг **11**

10 Дүгнэлт **11**

Ашигласан материал **11**

Хавсралт А. Ажлын төлөвлөгөө **12**

Хураангуй

Linux системийн процесс, thread-ийн мэдээллийг kernel module-оор уншиж, холбоосыг графаар харуулах TaskGraph програм хэрэгжүүлэв. Модуль нь PID, төлөв, нэрийг LINEAR болон DFS дарааллаар хэвлэж, `/proc/taskgraph` файлаар шинэ хэмжилт өгнө. UI-аас дэд мод сонгох, thread шүүх, дарааллаар үзэх, мэдээлэл хадгалах боломжтой. Python-ийн 28, бодит Linux kernel-ийн 17 шалгалт нийт 45/45 тэнцсэн. Туршилтын 7 task-ийн нэр, төлөв, parent болон thread group нь procps-ийн гаралттай таарсан.

1. Удиртгал

Зорилго нь Linux-ийн процессын бүтэц, эцэг-хүүхэд холбоо болон DFS аргыг бодит өгөгдөл дээр судлах юм. Процесс үүсэх, зогсох, дуусах өөрчлөлтийг граф, хүснэгтээр ажиглана. Хэрэгжилт нь C kernel module, Python parser, Tkinter UI гэсэн гурван хэсэгтэй.

2. Сурах бичгийн шаардлага

Operating System Concepts сурах бичгийн 3-р бүлгийн Programming Project 3 нь task-уудыг шугаман болон гүний хайлтаар жагсаахыг шаарддаг [1].

Хүснэгт 1: Үндсэн шаардлага ба хэрэгжилт

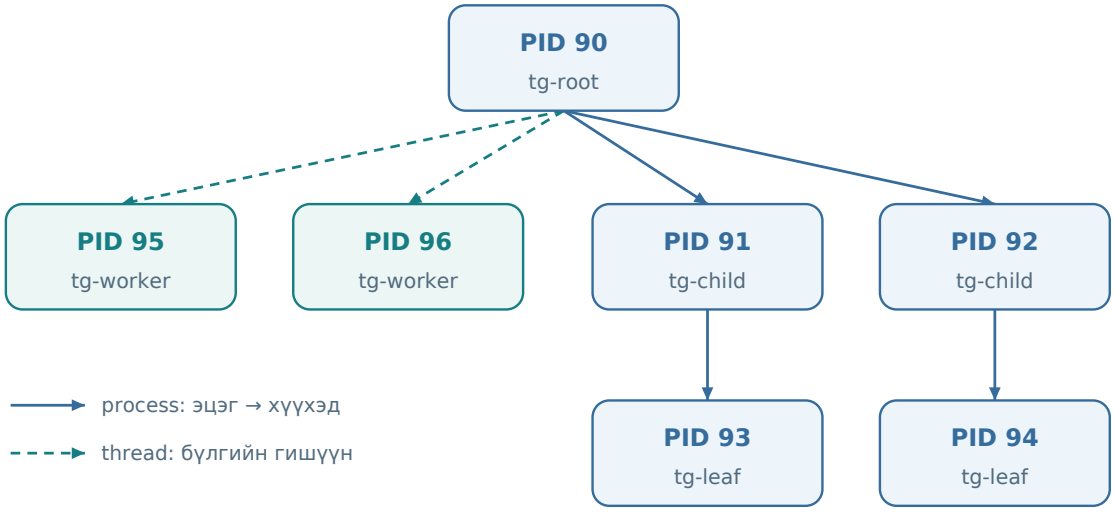
Шаардлага	Хэрэгжилт
Шугаман жагсаалт	`for_each_process()` ашиглаж process leader бүрийн PID, state, comm-ийг kernel log-д хэвлэнэ.
DFS жагсаалт	`init_task`-аас эхэлсэн модыг kernel дотор стекээр туулж, дараалал ба гүнийг гаргана.
Зөв ажиллагааг шалгах	Process жагсаалтыг `ps -el`, thread-үүдийг `ps -eLf`-тэй харьцуулж болно.
Task graph UI	Parent-child болон process-thread холбоосыг дүрсэлж, дахин уншихад шинэчилнэ.
Өөрчилж турших	Root PID, thread шүүлт, харагдах тоо, шинэчлэх хугацааг өөрчилнө. Буруу оролтыг мэдээлнэ.

Номын Part I-ийг `for_each_process()`, Part II-ийг `init_task`-аас эхэлсэн `children/sibling` холбоосоор шууд хэрэгжүүлсэн. Хүүхэд зангилааг `list_entry()`-ээр авч, стек дээр эцэг зангилааны iterator-ийг хадгална. Kernel stack дээр рекурс ашиглахгүй.

3. Онолын үндэс

Linux-ийн `task_struct` бүтэц нь нэг task-ийн PID, төлөв, нэр болон бусад task-тай холбогдох жагсаалтын холбоосыг агуулдаг. Процесс ба thread хоёулаа энэ бүтцээр дүрслэгддэг ч бүлэгт харьяалагдах байдлаараа ялгаатай. **PID** нь тухайн task-ийн дугаар, **TGID** нь thread group-ийн дугаар юм. Process leader-ийн PID ба TGID ижил; нэмэлт thread-ийн PID өөр боловч TGID нь leader-ийн PID-тай тэнцэнэ.

PPID нь эцэг процессын дугаарыг илэрхийлнэ. Гэвч thread-ийн бүлгийн гишүүнчлэл нь эцэг-хүүхдийн холбоотой адил биш. Иймээс графт process-ийг PPID руу нь, нэмэлт thread-ийг TGID руу нь холбоно. Бодит PPID-г тусдаа баганад хадгалснаар дүрслэхэд хэрэглэсэн `parent` утгыг үйлдлийн системийн эцэг процессын дугаартай андуурахгүй.



Бичигдсэн DFS дараалал: 90, 95, 96, 91, 93, 92, 94

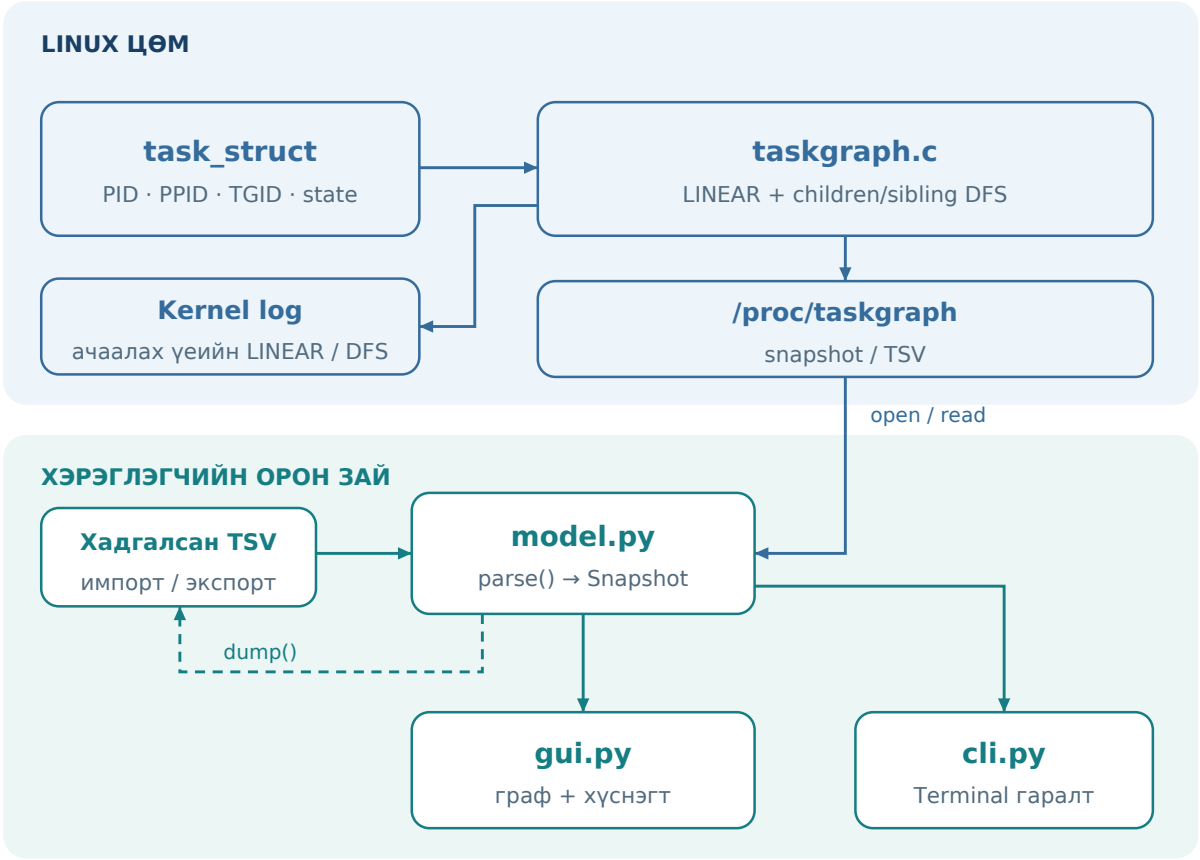
Зураг 1: Бодит created snapshot-ийн 7 task: бүтэн сум нь эцэг-хүүхэд, тасархай сум нь thread гишүүнчлэл.

Зураг 1-т PID 90 нь үндсэн процесс, PID 91, 92 нь түүний хүүхэд, PID 93, 94 нь дараагийн үеийн хүүхэд байна. PID 95, 96 нь TGID 90-той thread-үүд бөгөөд тусдаа хүүхэд процесс гэж тооцогдохгүй. **LINEAR** жагсаалт нь process leader-уудыг kernel-ийн process жагсаалтаар уншина. **DFS** нь эцгийг үзсэний дараа хүүхдийн дэд модыг бүрэн туулж, дараагийн ах дүү рүү шилжинэ. Манай өргөтгөл leader-ийн дараа түүний нэмэлт thread-үүдийг оруулдаг тул зурагт буй DFS дараалал 90, 95, 96, 91, 93, 92, 94 болсон.

Төлөвийн тэмдэг нь хэмжилтийн агшныг илэрхийлнэ: R — ажиллаж байгаа эсвэл бэлэн; S — унтаж байгаа; I — idle kernel thread; D — тасалдуулж болохгүй хүлээлт; T/t — зогсоосон; Z/X — zombie/dead. Дараагийн хэмжилтэд төлөв өөрчлөгдөж болно.

4. Системийн зохиомж

Системийг Linux цөмөөс мэдээлэл авах хэсэг, тэр мэдээллийг шалгаж дүрслэх хэрэглэгчийн хэсэг гэж зохион байгуулсан. Kernel module нь бодит task-ийн бүтэцтэй ажиллаж, Python програм нь module-ийн гаргасан өгөгдлийг уншина. Ингэснээр граф зурах, шүүлт хэрэглэх болон файл хадгалах үйлдэл kernel-ийн task жагсаалтыг өөрчлөхгүй.



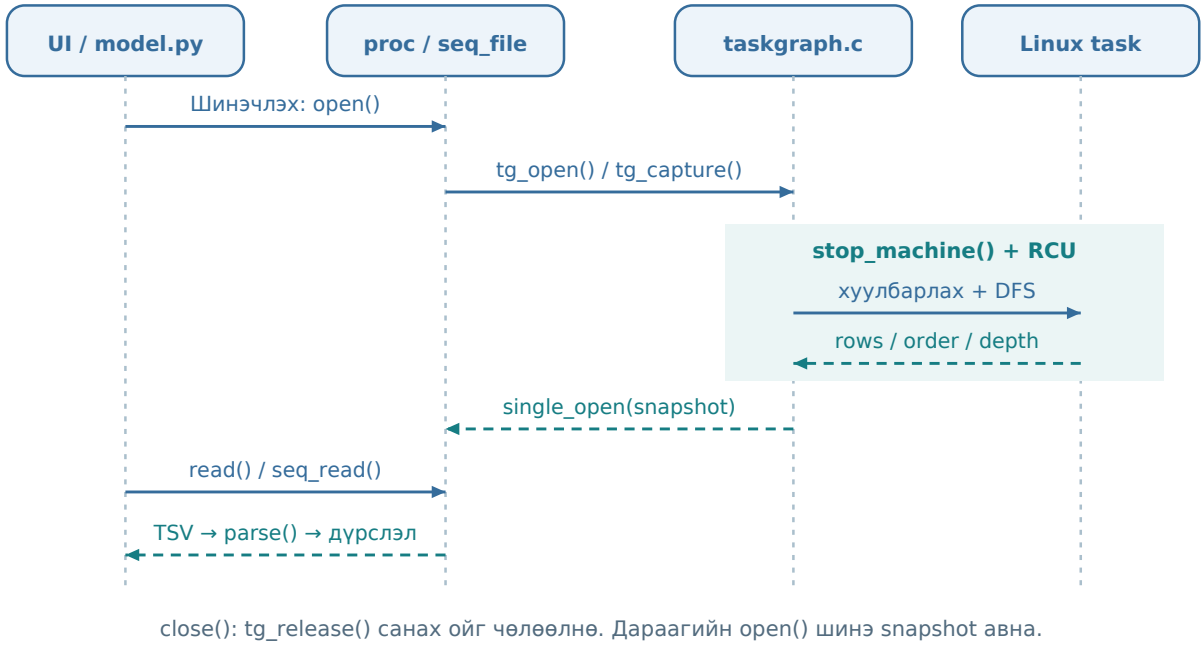
Зураг 2: TaskGraph-ийн архитектур ба өгөгдлийн урсгал. Цөм ба хэрэглэгчийн орон зайг тусад нь тэмдэглэв.

Зураг 2-ийн дээд хэсэгт `taskgraph.c` нь `task_struct`-уудаас өгөгдөл хуулж, LINEAR болон DFS дарааллыг гаргана. Module ачаалах үеийн жагсаалт kernel log-д хэвлэгддэг бол `/proc/taskgraph` нь файл нээх бүрд шинэ snapshot өгдөг интерфэйс юм. Иймээс эхний log ба дараа нь уншсан proc файл өөр агшны хэмжилт байж болно.

Доод хэсгийн `model.py` нь TSV өгөгдлийг parse хийж, шалгагдсан `Snapshot` объект үүсгэнэ. `gui.py` ба `cli.py` энэ ижил загварыг ашигладаг тул график болон командын мөрийн хувилбар тус тусдаа өгөгдөл тайлбарлах шаардлагагүй. TSV экспорт нь бүтэн snapshot-ийг хадгалж, дараа нь файл импортлох замаар ижил хэмжилтийг дахин шалгах боломж олгоно.

4.1. Snapshot унших дараалал ба интерфэйс

Шинэчлэх үйлдэл `/proc/taskgraph`-ийг нээж, `tg_open()`-оос `tg_capture()`-ийг дуудан snapshot үүсгэнэ. Зураг 3-ийн тасархай сум нь буцаах өгөгдөл, ногоон хэсэг нь CPU-уудыг түр саатуулж task мэдээллийг хуулж буй үеийг заана.



Зураг 3: Snapshot авах, унших, чөлөөлөх үйлдлийн дараалал.

Хүснэгт 2: Компонентуудын үүрэг

Компонент	Үүрэг
`taskgraph.c`	Шугаман жагсаалт, шууд children/sibling DFS, proc snapshot үүсгэнэ.
`model.py`	TSV өгөгдлийг шалгаж, дэд мод болон thread шүүлт хийнэ.
`gui.py`, `cli.py`	Нэг өгөгдлийг график болон командын мөрөөр үзүүлнэ.
`workload.py`	Өөр өөр гүн, салаалалттай туршилтын процессын мод үүсгэнэ.

Snapshot нь `pid`, `ppid`, `tgid`, `parent`, `state`, `kind`, `linear`, `dfs`, `depth`, `orphan`, `comm` баганатай. `linear`, `dfs` нь дараалал, `depth` нь гүн юм. Header-ийн `count`, `cap`, `truncated` утгаар хэмжилт бүтэн эсэхийг шалгана. Хэсэгчлэн уншихад snapshot өөрчлөгдөхгүй; файл хаахад санах ойг чөлөөлнө.

Root PID, Threads, DFS/LINEAR сонголтууд нь уншсан snapshot-д шүүлт, эрэмбэ хэрэглэнэ. *Шинэчлэх* болон авто шинэчлэлт эх үүсвэрийг дахин уншина. File горимд ижил файлыг дахин унших тул kernel-ийн шинэ өөрчлөлт тэр файлд автоматаар нэмэгдэхгүй.

5. Хэрэгжилт

`tg_capture()` нь мөр, стек, индексийн санах ойг урьдчилан хуваарилна. `stop_machine()` дотор CPU-уудыг түр саатуулж, процессын холбоос өөрчлөгдөхгүй үед мэдээллийг хуулна. Энэ нь module-д export хийгдээгүй `tasklist_lock`-ийг шууд ашиглахгүйгээр жагсаалтыг хамгаална [2]. RCU уншилтын хамгаалалтыг мөн хэрэглэнэ [3].

`tg_walk()` нь `init_task.children`-аас эхэлж, `sibling` холбоосоор хүүхдүүдийг гүнээр туулна. Thread бүрийн children жагсаалтыг шалгадаг тул process leader бус thread-ийн үүсгэсэн хүүхэд орхигдохгүй. Thread-үүдийг TGID-ийн гишүүнчлэлээр нэмнэ. Callback дотор санах ой хуваарилах, хэвлэх, унтуулах үйлдэл хийхгүй; модулийн entry point-ооc LINEAR, DFS үр дүнг kernel log-д хэвлэнэ. `seq_file` нь нэг нээлттэй файлаас хэсэгчлэн уншихад ижил snapshot ашиглана [4]. Дахин нээх бүр шинэ өгөгдөл авна. `max_tasks` нь 16–16384, анхны утга 4096. Хязгаар хүрвэл `truncated=1` гэж мэдээлнэ. Parser нь формат, PID давхардал, дараалал, төлөв болон оролтын хэмжээг шалгана.

GitHub-ийн main салбарт эх код, тест, баримтыг commit-оор хадгалсан. Түүхэнд хэрэгжилт, Ubuntu заавар, багийн мэдээлэл, тайлангийн засварууд тусдаа бүртгэлтэй. Хувилбар: v1.1.3-bd1.

6. Туршилт ба UI дүрслэл

Графын тасралтгүй сум нь process parent-child, тасархай сум нь thread гишүүнчлэлийг заана. Зангилаа сонгоход дэлгэрэнгүй харагдана.

Хүснэгт 3: Өөрчилж турших удирдлагууд

Удирдлага	Үйлдэл
DFS / LINEAR	Task үзүүлэх дарааллыг солино.
Root PID / Threads	Дэд мод сонгож, нэмэлт thread-үүдийг харуулна эсвэл нууна.
Зангилааны хязгаар	1–2000 хүртэл task харуулна; бүтэн snapshot-ийг өөрчлөхгүй.
Шинэчлэх / Авто шинэчлэлт	Эх үүсвэрийг дахин уншина. Интервал 500–10000 ms.
Өмнөх / Дараах / Тоглуулах	Нэг snapshot дотор traversal-ийг алхмаар харуулна.
Файл нээх / Хадгалах / Reset	Snapshot импортлох, экспортлох, view тохиргоо сэргээх.

Kernel горим бодит модулиас, File горим хадгалсан snapshot-оос, Fixture горим тестийн жишээнээс уншина. File горимын авто шинэчлэлт ижил файлыг дахин уншина.

7. Туршилтын арга

Хүснэгт 4: Туршсан орчин

Хэсэг	Орчин
Build	Ubuntu 24.04, GCC 13.3.0, Linux 6.8.0-31 headers.
Kernel ажиллагаа	QEMU 8.2.2 TCG, 2 vCPU, 512 MiB RAM, Linux 6.8.0-31-generic, initramfs.
UI, автомат тест	Python 3.12, Tk 8.6, unittest, Xvfb.

Python тестээр оролт шалгах, LINEAR/DFS, дэд мод, thread шүүлт, импорт/экспорт, буруу оролт болон 5000 гүн модыг шалгав. Kernel тестээр модулийг ачаалж, процесс ба thread үүсгэн `before`, `created`, `stopped`, `after`, `capped` үеийн snapshot авсан. C workload нь 5 process, нэмэлт 2 thread буюу 7 task үүсгэнэ. Procps-ng 4.0.4-ийн `ps -el`, `ps -eLf` гаралтыг хадгалж, PID, PPID, TGID, нэр, төлөвийг тусдаа баганатай гаралтаар тулгасан. Root-д SIGSTOP өгөхөд T төлөв болох, дуусгахад бүх хүүхэд цэвэрлэгдэхийг шалгасан. Үр дүнг `docs/evidence/` дотор log, JSON, TSV хэлбэрээр хадгалсан.

8. Үр дүн

Python-ийн **28/28 тест**, kernel-ийн **17/17 шалгалт** тэнцэв. Модуль амжилттай хөрвөж, Linux 6.8.0-31 дээр ачаалагдсан.

Хүснэгт 5: Бодит kernel дээрх шалгалтын үр дүн

Код	Ажигласан үр дүн	Төлөв
K1	Модуль ачаалагдав; `insmod` exit=0.	Тэнцсэн
K2	Kernel эх үүсвэртэй 66 task; PID 0 багтсан.	Тэнцсэн
K3	LINEAR болон DFS log хоёулаа хэвлэгдсэн.	Тэнцсэн
K4	73 task-ийн DFS rank давхцаагүй; parent нь child-ээс өмнө.	Тэнцсэн
K5	Root PID 90-ийн дэд мод 7 task-тай.	Тэнцсэн
K6	PID 95, 96 thread-үүд TGID 90-тай холбогдсон.	Тэнцсэн
K7	SIGSTOP дараа root-ийн state=T болсон.	Тэнцсэн
K8	Workload дууссаны дараа `tg-*` task үлдээгүй.	Тэнцсэн
K9	Модуль буухад `/proc/taskgraph` устсан.	Тэнцсэн
K10	`max_tasks=1` буруу утгыг хүлээж аваагүй.	Тэнцсэн
K11	`max_tasks=16`: count=16, truncated=1.	Тэнцсэн
K12	Дахин ачаалах, буулгах үйлдэл амжилттай.	Тэнцсэн
K13	children/sibling шууд traversal ашигласан snapshot.	Тэнцсэн
K14	Туршилтын 7 task procps-ийн гаралттай таарсан.	Тэнцсэн
K15	Процесс үүсэх/дуусах үед 10 удаа snapshot уншсан.	Тэнцсэн
K16	Kernel log-д BUG, Oops, WARNING, lockup гараагүй.	Тэнцсэн
K17	Thread-ээс үүсгэсэн хүүхэдтэй 8 task бүрэн харагдсан.	Тэнцсэн

Хүснэгт 6: UI болон өгөгдлийн боловсруулалтын шалгалт

Туршилт	Хүлээгдсэн ба гарсан үр дүн
Жишээний DFS	0, 1, 10, 11, 20, 21, 2.
Root PID=10	10, 11, 20, 21 гэсэн 4 task харагдсан.
Threads нуух	PID 11 нуугдсан.
Байхгүй PID, буруу хязгаар	Ойлгомжтой алдаа гарсан.
Импорт/экспорт	Snapshot-ийн task өгөгдөл өөрчлөгдөөгүй.

Kernel тестийн дэлгэрэнгүй нь `kernel_results.json`, serial log нь `qemu-serial.log`, Python тестийн гаралт нь `docs/unit-tests.log` файлд бий. PID нь дахин ажиллуулахад өөрчлөгдөх тул тест root PID-г тухайн run-ооc уншдаг.

8.1. `ps` командтай харьцуулсан үр дүн

Харьцуулалтыг Linux 6.8.0-31 дээр procps-ng 4.0.4-өөр гүйцэтгэсэн. Workload ажиллаж байх үед модулийг дахин ачаалж entry point-ийн LINEAR, DFS log-ийг авсан. Дараа нь snapshot болон номын хоёр командыг хадгалсан. PID, нэр, төлөв, холбоосыг parse хийхийн тулд нэмэлт `ps -e -o ...`, `ps -eL -o ...` гаралт ашиглав.

Хүснэгт 7: Бүх системийн жагсаалтын харьцуулалт

Үзүүлэлт	LINEAR	DFS	Тайлбар
Entry log-ийн task	70	73	DFS-д PID 0 орно.
`ps -el` / `ps -eLf`	70	72	PID 0 орохгүй.
Log ба ps-д нийтлэг ID	69	71	Түр task ялгаатай.
Snapshot-ийн нийтлэг ID	69	71	PID 0-ийг хассан.
PPID, TGID таарсан	69/69	71/71	Бүх нийтлэг ID.
Төлөв яг таарсан	67/69	69/71	Хэмжилтийн агшин өөр.
Богино нэр яг таарсан	54/69	56/71	Kernel нэрийн урт ялгаатай.

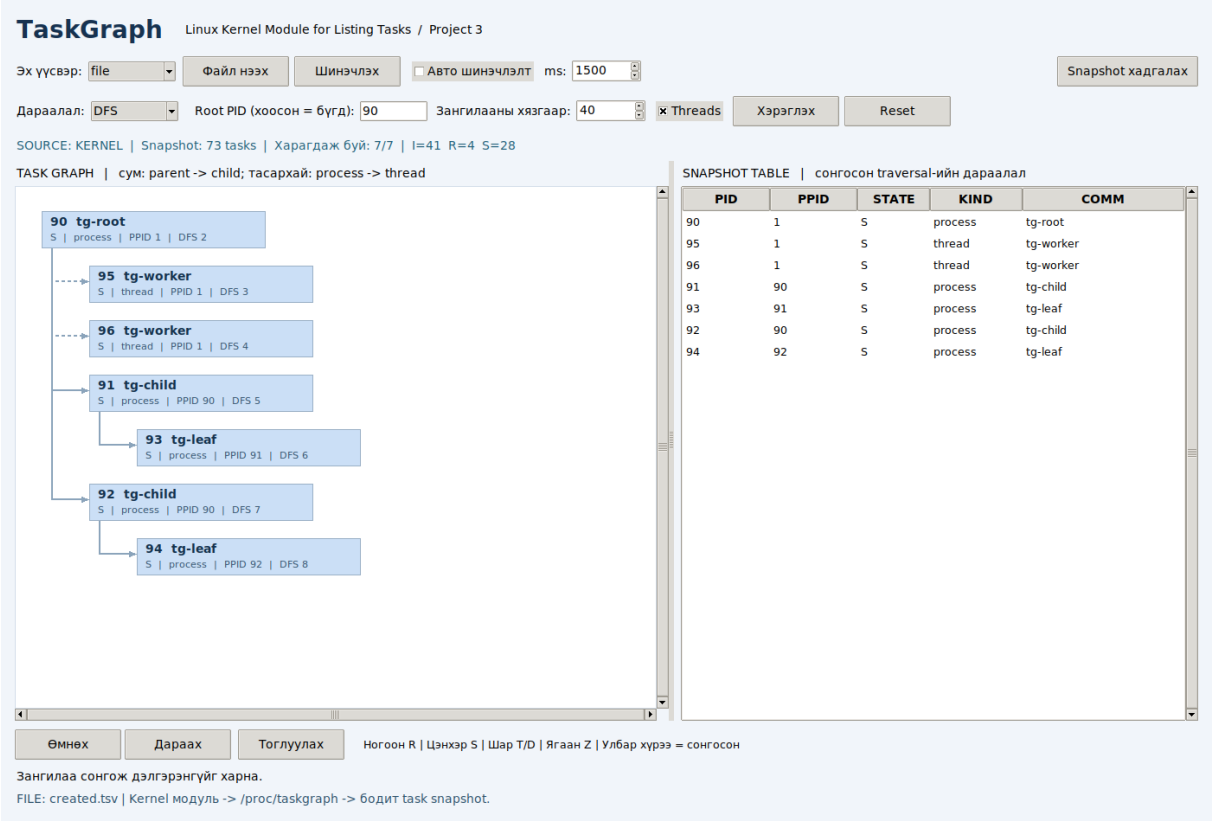
Snapshot-д PID 102 нь `cat`, харин нэмэлт ps хэмжилтэд PID 106, 107 нь `ps` байв. Entry log-д тухайн үеийн `insmod`, raw ps гаралт бүрт өөрийн `ps` процесс ордог. Эдгээр нь дараалсан хэмжилтийн хүлээгдсэн зөрүү юм. Kernel thread-ийн урт болон нэмэлт тайлбартай нэрийг procps дэлгэрүүлж харуулдаг; модуль `TASK_COMM_LEN`-ийн богино нэр авна. Төлөвийн хоёр зөрүү нь ажиллаж буй kernel task-ууд хэмжилтийн хооронд өөрчлөгдсөнөөс үүссэн.

Хүснэгт 8: Тогтвортой workload-ийн нарийвчилсан тулгалт

PID/LWP	PPID	TGID	Нэр	Төлөв
90	1	90	`tg-root`	S = S
91, 92	90	91, 92	`tg-child`	S = S
93	91	93	`tg-leaf`	S = S
94	92	94	`tg-leaf`	S = S
95, 96	1	90	`tg-worker`	S = S

Туршилтын бүх 7 task-ийн нэр, төлөв, PPID, TGID таарсан. 5 process LINEAR log, `ps -el`-д; бүх 7 task DFS log, `ps -eLf`-д багтсан. Thread-ийн бодит PPID=1, харин графын parent=TGID=90 болохыг ялгаж шалгасан. Дэлгэрэнгүй: `docs/evidence/ps_comparison.json`, `ps-el.txt`, `ps-eLf.txt`, `comparison-kernel.log`.

8.2. Процесс ба thread-ийн граф



Зураг 4: Kernel-ийн created snapshot-ийг UI-д нээсэн байдал.

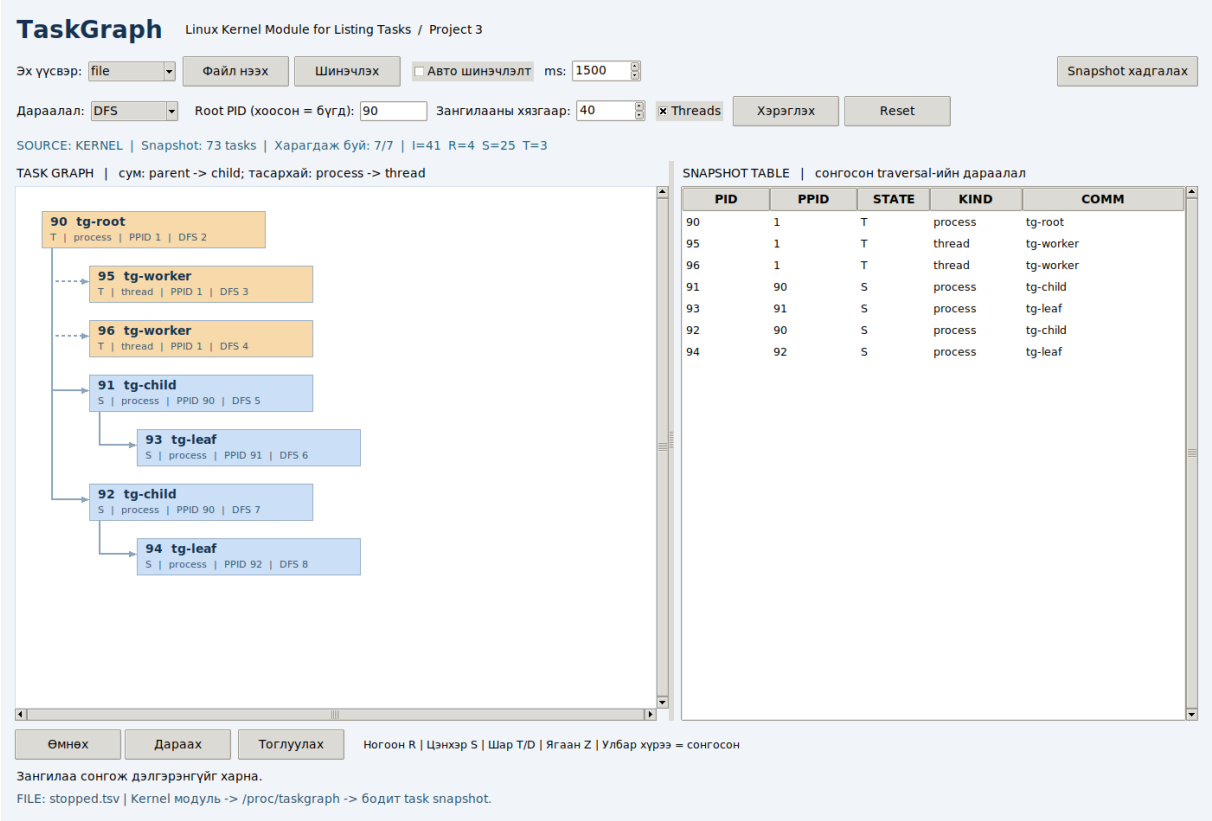
Root PID 90-ийн дэд модонд 7 task байна. PID 95, 96 нь thread тул тасархай сумаар холбогдсон. PID 91, 92 нь root-ийн хүүхэд; PID 93, 94 нь тэдгээрийн хүүхэд юм.

Хүснэгт 9: Зурагт харагдаж буй task-ууд

PID	Графын parent	Төрөл	Нэр
90	1	Process	`tg-root`
95, 96	90	Thread	`tg-worker`
91, 92	90	Process	`tg-child`
93, 94	91, 92	Process	`tg-leaf`

Зургийн өгөгдлийг QEMU дахь бодит Linux kernel module-оос авч, `created.tsv` файлаар UI-д импортолсон. Графын холбоос, хүснэгтийн мөрүүд нэг snapshot-д тулгуурлана.

8.3. Төлөвийн өөрчлөлт



Зураг 5: SIGSTOP дараах snapshot: root PID 90-ийн төлөв T болсон.

SIGSTOP өгсний дараа PID 90 шар өнгөөр дүрслэгдэв. SIGCONT өгч үргэлжлүүлэн, workload-ийг дуусгахад үүсгэсэн процесс, thread-үүд цэвэрлэгдсэн. Хязгаарласан snapshot-д `truncated=1` тэмдэг ил тод гарсан.

Хүснэгт 10: Хэмжилтүүдийн харьцуулалт

Үе шат	Үр дүн
Created	Root-ийн дэд мод: 5 process + 2 thread.
Stopped	Root-ийн state=T.
After	Workload-ийн task үлдээгүй.
Capped	16 мөр; snapshot дутууг тэмдэглэсэн.
Unloaded	Proc файл устсан.

9. Хэлэлцүүлэг

Граф болон хүснэгт нь kernel-ийн PID, parent, төлөвийн мэдээлэлтэй тохирч байв. Тестээр process-thread ялгалт, DFS дараалал, STOP төлөв, буруу параметр болон модулийн цэвэрлэгээг шалгасан. `stop_machine()` ашигласнаар бүтэц өөрчлөгдөхгүй үед children/sibling холбоосыг уншина. Гэхдээ CPU-уудыг түр саатуулдаг тул энэ хэрэгжилт сургалтын Linux VM-д зориулагдсан; бага latency шаардсан системд тохиромжгүй. Snapshot хооронд PID, төлөв өөрчлөгдөнө. Хэмжээний хязгаараас болж parent багтаагүй бол root-д холбоод `orphan=1` гэж тэмдэглэнэ. `comm` нь богино нэр; бүтэн командын мөр биш. Том системд `max_tasks`-ийг өсгөнө. Kernel ажиллагааг QEMU initramfs орчинд шалгасан. Тайлангийн UI зураг нь тэндээс авсан snapshot-ийн дүрслэл юм. Хадгалсан snapshot нь өмнөх хэмжилтийг давтан үзэхэд зориулагдсан; бодит Linux хэмжилтийг `--kernel` горимоор ажиллуулна.

10. Дүгнэлт

Процессын жагсаалтыг LINEAR болон DFS аргаар гаргах kernel module, task graph UI, snapshot хадгалах болон дахин үзэх боломжийг хэрэгжүүлэв. Нийт 45 автомат шалгалт тэнцсэн. Номын хоёр traversal болон procps-тэй харьцуулах шаардлагыг туршилтын нотолгоогоор баталгаажуулсан. Төслөөр Linux-ийн task бүтэц, process-thread холбоо, төлөв болон DFS-ийг бодит өгөгдөлтэй холбон судалсан.

Ашигласан материал

[1] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*. 10th ed., Wiley, 2018. Chapter 3, Programming Project 3, P-17–P-19.

[2] Linux 6.8. *stop_machine API, include/linux/stop_machine.h*. Kernel header-ийн API тайлбар.

[3] Linux 6.8. *Using RCU to Protect Read-Mostly Linked Lists*. https://docs.kernel.org/6.8/RCU/listRCU.html

[4] Linux 6.8. *The seq_file Interface*. https://docs.kernel.org/6.8/filesystems/seq_file.html

[5] Linux 6.8. *Building External Modules*. https://docs.kernel.org/6.8/kbuild/modules.html

Код, тест, баримт бичгийг боловсруулахад ChatGPT/Codex-ийн туслалцаа ашигласан.

Хавсралт А. Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	IV-2-3
Төсөл	Linux Kernel Module for Listing Tasks
Repository	https://github.com/Temka0511/FCSA_302-IV-2-3-LINUX-Kernel-Module-For-Task-Information
Хэл/сан/орчин	C, Python 3, Tkinter; Ubuntu 24.04 LTS, Linux 6.8, GCC/Make
Үндсэн дүрслэл	Процессын parent-child болон process-thread холбоосын граф

Гишүүн	Хариуцах ажил	Commit/file
А.Тэмүүлэн B242270063	Kernel module, LINEAR болон children/sibling DFS хэрэгжилт	src/kernel/taskgraph.c src/kernel/Makefile
Ч.Балгандамба B232270142	Snapshot формат, parser, оролтын шалгалт, нэгж тест	src/taskgraph/model.py tests/test_model.py
Д.Азмаа B241940101	Процессын граф, хүснэгт, шүүлт болон UI удирдлага	src/taskgraph/gui.py src/taskgraph/cli.py
Н.Билэгт-Очир B231910009	Workload, STOP/CONT туршилт, ps харьцуулалт	tests/guest_workload.c scripts/verify_live.py
Т.Түвшинтөр B190910820	Орчин, build/run, QEMU болон boundary/error шалгалт	scripts/check_env.sh scripts/qemu_smoke.py
Б.Өсөхбаяр B210930901	Тайлан, хэрэглэгчийн гарын авлага, туршилтын нотолгоог нэгтгэх	docs/latex/report.tex docs/latex/manual.tex

Тест	Оролт/нөхцөл	Хүлээгдсэн үр дүн
1	Build, insmod; max_tasks=4096, log_on_load=1	Модуль ачаалагдаж, proc файл болон LINEAR/DFS log үүснэ.
2	5 process + 2 thread бүхий workload; ps -el, ps -eLf	7 task-ийн PID, PPID, TGID, нэр, төлөв таарна.
3	Жишээ файл; Root PID=10; Threads асаах/унтраах	4 task харагдана; thread нуухад 3 process үлдэнэ.
4	Workload-ийн root-д SIGSTOP, SIGCONT өгөх; дараа нь дуусгах	T төлөвт орж, үргэлжлүүлсний дараа хэвийн ажиллана; дуусвал жагсаалтаас алга болно.
5	max_tasks=1, дараа нь 16; Root PID-д abc оруулах	Буруу сар, PID-г мэдээлнэ; сар=16 үед truncated=1 гарна.